Creating MongoDB Indexes

Name: Zedekiah Heteroza

Student ID: 24-0521-842

Step 1: Database Setup

```
use mydatabase
db.createCollection("products")

db.products.insertMany([
{ name: "Laptop", category: "Electronics", price: 1200, tags: ["new", "featured"] },
{ name: "Smartphone", category: "Electronics", price: 800, tags: ["popular", "sale"]
},
{ name: "T-shirt", category: "Clothing", price: 25, tags: ["new", "discount"] },
{ name: "Jeans", category: "Clothing", price: 75, tags: ["popular"] },
{ name: "Coffee Maker", category: "Appliances", price: 100, tags: ["new"] },
{ name: "Blender", category: "Appliances", price: 50, tags: ["sale"] },
{ name: "Tablet", category: "Electronics", price: 300, tags: ["featured"] },
{ name: "Dress", category: "Clothing", price: 60, tags: ["discount"] },
{ name: "Microwave", category: "Appliances", price: 150, tags: ["popular"] },
{ name: "Headphones", category: "Electronics", price: 120, tags: ["new", "sale"] }
])
```

Step 2: Query Performance Without Index

```
db.products.find({ category: "Electronics"}).explain("executionStats")
```

```
mydatabase> db.products.find({ category: "Electronics" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'mydatabase.products',
    parsedQuery: { category: { '$eq': 'Electronics' } },
    indexFilterSet: false,
    queryHash: 'A248FCC2',
    planCacheShapeHash: 'A248FCC2',
    planCacheKey: 'FFD5D11A',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { category: { '$eq': 'Electronics' } },
      nss: 'mydatabase.products',
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { category: { '$eq': 'Electronics' } },
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 11,
      advanced: 4,
      needTime: 6,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'mydatabase.products',
      direction: 'forward',
      docsExamined: 10
    }
  },
```

Before indexing, MongoDB performed a collection scan (COLLSCAN), meaning all documents were checked. This resulted in a zero-millisecond execution time and more documents examined.

Step 3: Single Field Index

```
db.products.createIndex({ category: 1 })
```

Step 4: Query With Single Index

```
db.products.find({ category: "Electronics" }).explain("executionStats")
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 7,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'mydatabase.products',
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'mydatabase.products',
      keyPattern: { category: 1 },
      indexName: 'category_1',
      isMultiKey: false,
      multiKeyPaths: { category: [] },
      isUnique: false,
      isSparse: false,
```

After creating the index, MongoDB used IXSCAN instead of COLLSCAN. The number of documents examined significantly decreased, and execution time remained at zero ms, showing improved performance.

Step 5: Compound Index

```
db.products.createIndex({ category: 1, price: 1 })
```

Step 6: Compound Query Performance

```
db.products.find({
  category: "Electronics",
  price: { $gt: 500 }
}).explain("executionStats")
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 0,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 4,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    nss: 'mydatabase.products',
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 1,
      nss: 'mydatabase.products',
      keyPattern: { category: 1, price: 1 },
      indexName: 'category_1_price_1',
      isMultiKey: false,
      multiKeyPaths: { category: [], price: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        category: [ '["Electronics", "Electronics"]' ],
        price: [ '(500, inf]' ]
      },
```

The compound index improved performance further by allowing MongoDB to filter using both category and price simultaneously, reducing scanned documents.

Step 7: Multi-Key Index (Array Field)

```
db.products.createIndex({ tags: 1 })
```

Step 8: Query Using Multi-Key Index

```
db.products.find({ tags: "new" }).explain("executionStats")
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 33,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 4,
    executionTimeMillisEstimate: 32,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    nss: 'mydatabase.products',
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 32,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 1,
      nss: 'mydatabase.products',
      keyPattern: { tags: 1 },
      indexName: 'tags_1',
      isMultiKey: true,
      multiKeyPaths: { tags: [ 'tags' ] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { tags: [ '["new", "new"]' ] },
```

The multi-key index improved query performance for array searches by allowing direct lookup of array values instead of scanning all documents.

Step 9: List and Drop Indexes

```
db.products.getIndexes()
db.products.dropIndex("category_1")
db.products.dropIndexes()
```

```
mydatabase>  db.products.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { category: 1 }, name: 'category_1' },
  { v: 2, key: { category: 1, price: 1 }, name: 'category_1_price_1' },
  { v: 2, key: { tags: 1 }, name: 'tags_1' }
]
mydatabase> db.products.dropIndex("category_1")
{ nIndexesWas: 4, ok: 1 }
mydatabase>  db.products.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { category: 1, price: 1 }, name: 'category_1_price_1' },
  { v: 2, key: { tags: 1 }, name: 'tags_1' }
]
mydatabase> db.products.dropIndexes()
{
  nIndexesWas: 3,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```